

The HMRC REST Components – Access and Refresh Tokens.

Author Ian H
Created 12/02/21
version 1.0

Introduction

The HMRC MTD REST APIs all use OAuth2 authentication, which requires the grant and use of access tokens. This was partly covered in the original “UsingTheDemos”, which was written for the original release of the VAT component and covered both the underlying components and using the VAT component specifically. Now with support for other APIs as well as VAT, the information relating to tokens has been given its own space.

OAuth2

In Wikipedia, see link below, Oauth is defined as an open standard for access delegation, commonly used as a way for internet users to grant applications access to their information without giving them the passwords. The HMRC MTD APIs use OAuth2, but it is not necessary to understand OAuth2 in order to use the UKDG REST client components. It does, however, help to understand that OAuth2 generates access tokens and in this case also refresh tokens, which the application should be able to store and load on demand.

The HMRC OAuth2 process

To start with, when there is no access token, the user will be taken to a login form. On the successful login, they will be asked to grant authority to let the application access the MTD APIs on their behalf. Clearly the person, or business granting access has to have an account with HMRC for the relevant service.

The grant process requires the Client ID and Client Secret created for the application, when it was registered with HMRC. It also requires a registered Callback Url, which should have been done as part of the registration process. If in doubt, check the HMRC on-line docs and look at the demos.

If the grant is successful, then the HMRC system returns an access token which conveys the authority for the application to access the relevant HMRC services on behalf of the grantor. The service also returns a refresh token and the expiry dates/times for both of these tokens.

Every call the application makes to the MTD API must be accompanied by a valid, current access token, without which the call will fail. The access token in this case is valid for 4 hours, or so it has always been up to this point. After 4 hours any call made with this access token will fail.

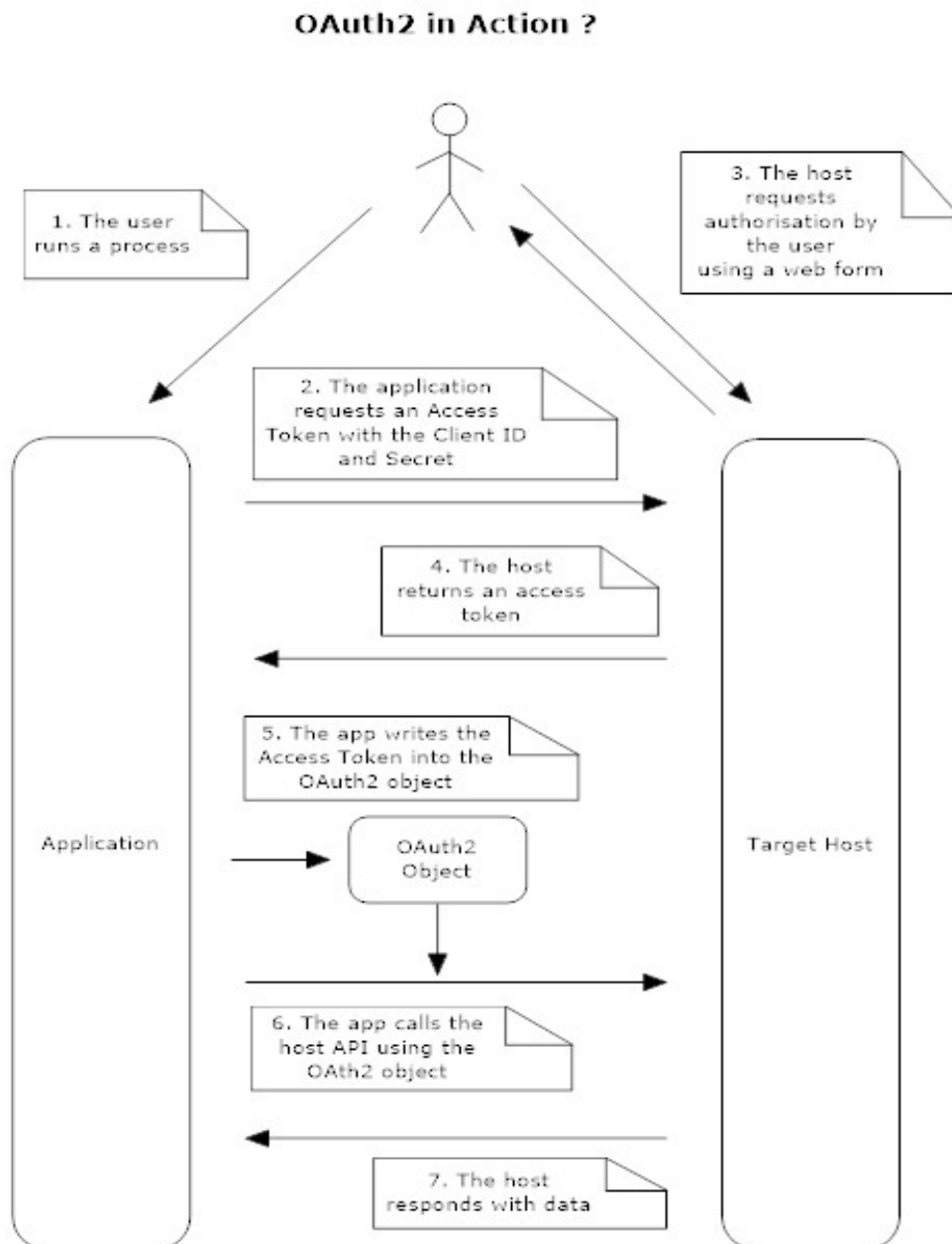
Access tokens are granted within the context of a scope and each MTD endpoint is related to a scope. If working with more than one endpoint, it may be necessary to get access granted for more than one scope, depending on the endpoints. Each scope will have separate and distinct access tokens.

While access tokens are only valid for 4 hours, the grant of the authority is valid for 18 months, during which time it is not necessary to go through the grant process again, as long as the tokens are managed effectively. Within the 18 month grant period, expired access tokens can be replaced by using the accompanying refresh token. A successful call to update with a refresh token results in the return of both a new access token and a new refresh token, together with the new expiry dates and

times. A refresh token can only update the access token which it originally accompanied.

A refresh token can be used at any time during the 18 months of the grant period, but it can only be used **once**. Any attempt to use it a second time will fail. When used, it is replaced by a new refresh token that also can only be used once, at any time during the remaining grant period. The refresh process can be used on an unlimited basis during the grant period, each time returning new access and refresh tokens.

At the end of the 18 month grant period, the refresh process will no longer work and it is necessary to go through the grant process again, to start another 18 months.



Refresh Token Issues.

HMRC noted that there had been much more trouble with refresh tokens than they had expected, so it is worth a paragraph here. Authorisation creates both an access token and a refresh token, but if they are not stored, then it will be necessary to go through the grant process each time.

The main issue seems to arise in situations where more than one computer is used to access the HMRC services. For example, where a normal process refreshes and saves tokens, but someone then accesses the service from another application/location and has to do the whole authorisation process, because they do not have access to the current tokens, this will then invalidate the tokens used by the normal process. The only option then is to repeat the whole authorisation process in the normal app.

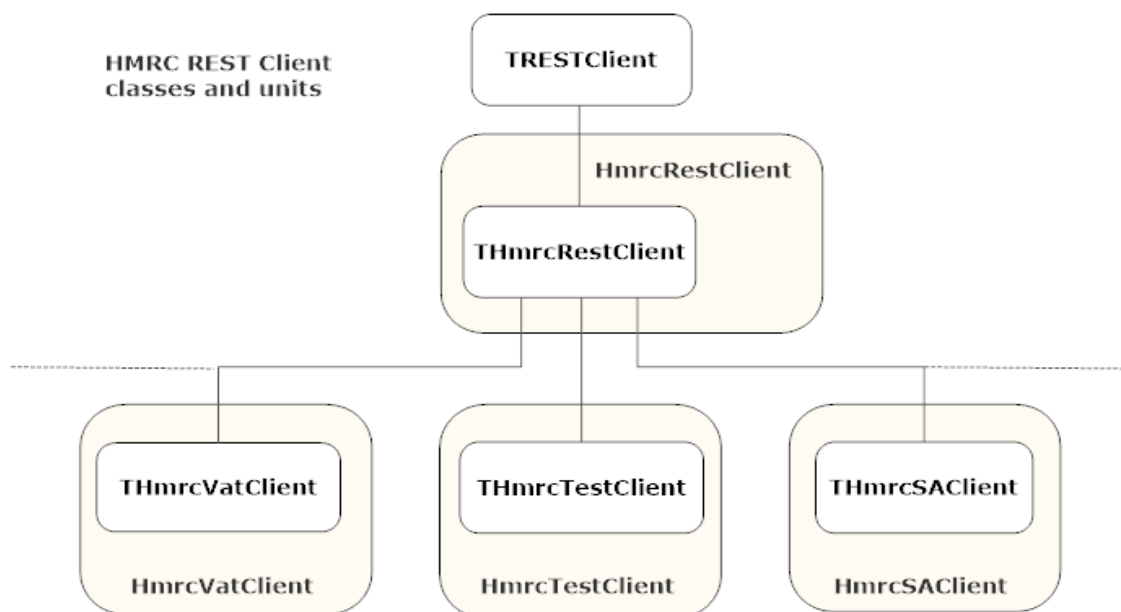
So a second authorisation grant process will invalidate the tokens generated by the previous grant action, even if it is within the original 18 month grant period. It will have cancelled the first grant period and started a second.

The UKDG MTD RestClient Components.

Resulting from a collaborative effort, towards then end of 2018, by a few members of the UKDG with an interest in handling the new MTD VAT API, the first release of the VATClient component was in early 2019. The first update was in December 2020, with another in January 2021 and a third in February 2021, including the new self assessment (SA) Client component.

From the start the rest components have handled both the grant process and the subsequent refreshes, assuming the application programmers provided the storage and loading capabilities. Using the rest components, the application developer does not really have to know anything about the Oauth process, unless they wish to.

The base class for the UKDG components, THMRCRestClient inherits from TRESTClient, so requires Delphi 10.0 (Seattle) probably as the entry level version with which it will work. TRESTClient is a comprehensive class to handle the whole REST process and includes the OAuth2 capabilities used. The methods to handle this for MTD are in the THMRCRestClient class, including the initial grant and subsequent refreshing.



The base class has a UID property for the current ID, which for VAT will be a VRN (VAT Registration No) and for Self Assessment it will be a UTR (Unique Taxpayer Reference). It also has a list for holding access tokens, defined in the HmrcRestSupport unit and 2 methods to add access

tokens to the list. These are **AddaToken** and **SetaToken**, the difference being that AddaToken will create a new token and add it to the list, as long as a token for the same user and scope does not exist, while SetaToken will check the list for an existing token for the scope alone, updating it with the new values if found and will only create a new token if one for this scope does not already exist.

Procedure AddaToken(Const uid, scp, atn, rtn: String; Const exp, tmo: TDateTime);

Procedure SetaToken(Const scp, uid, atn, rtn: String; Const exp, tmo: TDateTime);

The 2 token assignment methods allow for 2 different approaches to token management now. The original way, with AddaToken, simply loaded all saved tokens into the tokens list at the start and the rest client would search the list for a matching token each time. With SetaToken, the list can be managed in a different way, by maintaining only 1 token for each scope, updating it on demand when the UID is changed. This can be handled either by the application responding to something like an edit box exit event, or by setting a handler for the DoLoadTokens event.

Tokens can be loaded in bulk on start up, or dynamically on demand and it is up to the application developer to decide which to use. Examples of both are to be found in the various demo apps. The rest client will try to find a token in the tokens list when either the UID is changed or the scope is changed. If no token is found in the list and a method has been passed to the DoLoadTokens event, then it will try to load using that method.

The demos use a simple ini file to store the tokens, but this is just for demo purposes. Real applications can use any appropriate storage mechanism. The rest client components have no direct connection to the storage, which should be handled by the application.

Sample code snippet from a demo, where the application triggers a load on demand for a UID, using a fixed scope and typically triggered by an edit box exit event, or similar.

```
...
lvUid := edtUID.Text;
if (f.SectionExists(lvUid)) then
begin
  lvScope := 'read:vat';
  lvAccess := f.ReadString(lvUid, lvScope + '_Access', '');
  lvRefresh := f.ReadString(lvUid, lvScope + '_Refresh', '');
  lvTemp := f.ReadString(lvUid, lvScope + '_Stops', '');
  lvStops := StrToDateDef(lvTemp, Date);
  lvTemp := f.ReadString(lvUid, lvScope + '_Expires', '');
  lvExpires := StrToDateTimeDef(lvTemp, Now);

  OTestClient.AddaToken(lvUid, lvScope, lvAccess, lvRefresh, lvStops, lvExpires);
...

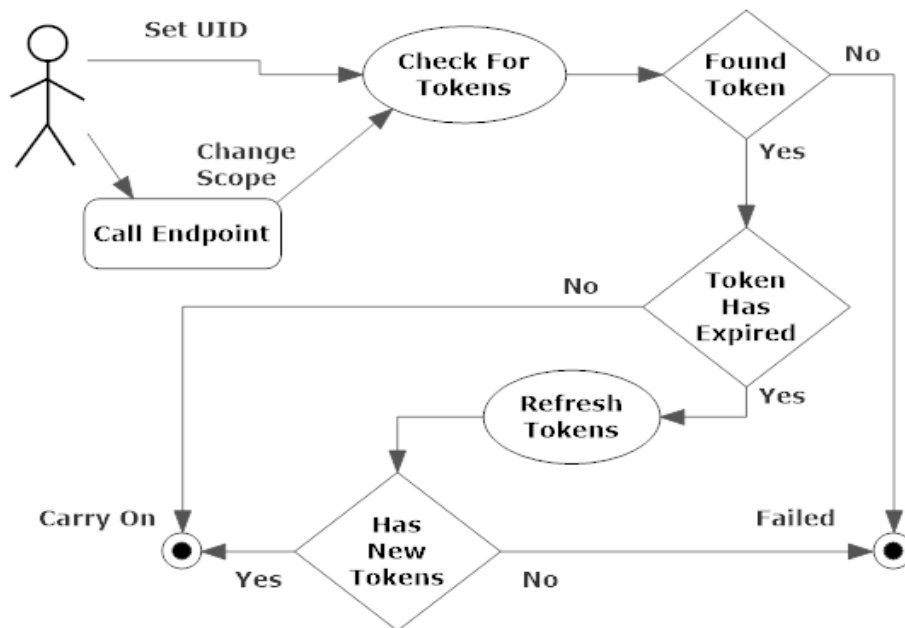
```

The method passed to the DoLoadTokens event should match the signature of the THmrcTokenLoadEvent type defined in the HmrcRestSupport unit. Similarly, any method passed to the OnTokenChange event should match the signature of the THmrcTokenEvent type from the same unit. This method should handle saving new and updated tokens.

THmrcTokenEvent = **procedure**(Sender: TObject; **const** uid, scp, atn, rtn: string; **const** exp, tmo: TDateTime) **of object**;

THmrcTokenLoadEvent = **procedure**(Sender: TObject; **const** uid, scp: string) **of object**;

There are separate components, SA, Test & VAT, which inherit from THMRCRestClient and implement methods to call specific endpoints within the APIs. When an endpoint is called, the client will automatically set the correct scope for that call. It should never be necessary for the application developer to set the call scope.



When an endpoint is called, the rest client checks for a valid token for the current UID and scope and just carries on if one is found. If an expired access token is found, then it will attempt to refresh it and, if successful, will just continue. If no access token is found, or the 18 month grant period has expired, then it will be necessary to go through the grant process (again).

The rest client does not automatically trigger the grant process itself, but the demos show how that can be automated within the application by checking the failure code value. It will, however, be necessary to call the endpoint again, after authority has been granted.

Sample code snippet from an application error handler.

```

...
// check whether it failed because there was no access token
if (OTestClient.LastCode = ERR_NO_ACCESS_TOKEN) then
begin
  // so get a new access token
  OTestClient.NewAccessToken;
  // now the user needs to call the api again
  mmoInfo.Lines.Add('Please call the API again, after authority has been granted');
end // if no token
...

```

The OnTokenChange event, if set, should be a pointer to an application method to save new or updated tokens. It will be called automatically when either a new access token is created by the grant process, or when a token is refreshed. The demos show examples of this process in action.

Some Links

Oauth – Wikipedia

<https://en.wikipedia.org/wiki/OAuth>

The basic connection tutorials were here :

<https://developer.service.hmrc.gov.uk/api-documentation/docs/tutorials>

Authorisation errors were here. This could be reached from the callback url page.

<https://developer.service.hmrc.gov.uk/api-documentation/docs/authorisation/user-restricted-endpoints#installed-applications>.